

Thread-Safe FIFO in C

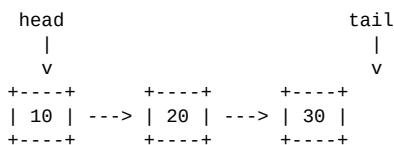
Complete theory, userspace implementation, producer-consumer synchronization, circular-buffer mechanics, and Linux kernel driver concepts

Interview objective: Be able to implement the FIFO, explain why every synchronization primitive exists, identify race conditions, and discuss how the design changes between userspace and Linux kernel code.

1. What is a FIFO?

FIFO means **First In, First Out**. The first item inserted is the first item removed. A queue therefore has two logical ends: producers insert at the **tail**, and consumers remove from the **head**.

```
push(10)
push(20)
push(30)
```



```
pop() returns 10
next pop() returns 20
```

2. Why does a FIFO need synchronization?

A normal queue is not automatically safe when multiple threads use it. A producer may update the tail while another producer is also updating it; a consumer may change the head while another consumer does the same; and producers and consumers may race while updating the shared count. These are data races and can corrupt queue state.

The shared state normally includes:

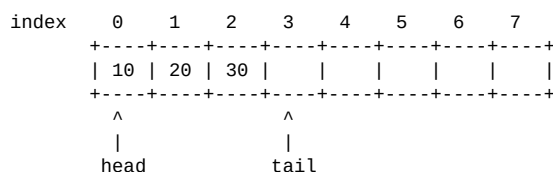
- **buffer:** storage containing the elements
- **head:** index of the next element to remove
- **tail:** index where the next element will be inserted
- **count:** number of valid elements currently in the FIFO

All operations that must observe or change this state consistently form a **critical section**.

3. Why use a bounded circular buffer?

A bounded FIFO uses a fixed-size array instead of allocating one linked-list node for every operation. This gives predictable memory usage, $O(1)$ insertion/removal, good cache locality, and no malloc/free in the hot path - useful for embedded and systems code.

```
FIFO_SIZE = 8
```



```
count = 3
```

The array is treated as circular. When an index reaches the final slot, the next operation wraps it back to zero:

```

tail = (tail + 1) % FIFO_SIZE;
head = (head + 1) % FIFO_SIZE;

For FIFO_SIZE = 8:

6 -> 7
7 -> 0
0 -> 1

```

The old value does not need to be erased after a pop. The metadata - head, tail, and count - determines which slots are logically valid.

4. Why keep count?

If head == tail, that state can mean either empty or full in a simple circular buffer. Keeping count makes the state explicit: count == 0 means empty, and count == FIFO_SIZE means full. An alternative design can sacrifice one slot and infer state from indices, but count is very clear in interviews.

5. Mutex: what does it protect?

In the userspace implementation, a **pthread_mutex_t** protects the queue state. Only one thread at a time may execute the protected queue manipulation. This prevents two producers or consumers from concurrently changing head, tail, count, or related buffer entries.

```

pthread_mutex_lock(&q->lock);

/* critical section */
q->buffer[q->tail] = value;
q->tail = (q->tail + 1) % FIFO_SIZE;
q->count++;

pthread_mutex_unlock(&q->lock);

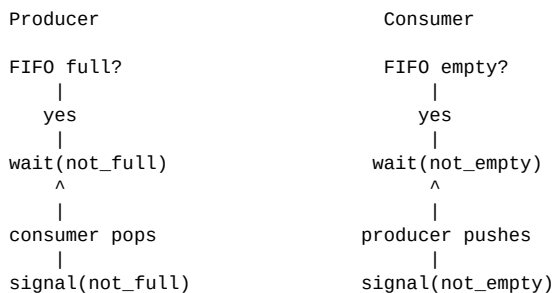
```

A mutex alone prevents corruption, but it does not solve efficient waiting. If a consumer finds an empty FIFO, repeatedly checking it in a loop wastes CPU. If a producer finds a full FIFO, repeatedly polling for free space has the same problem.

6. Condition variables: why are two needed?

A condition variable lets a thread sleep until some queue state may have changed. There are two independent conditions:

- **not_empty**: consumers wait here when count == 0. A producer signals it after adding an element.
- **not_full**: producers wait here when count == FIFO_SIZE. A consumer signals it after removing an element.



7. What exactly does pthread_cond_wait() do?

This is one of the most important interview questions. The caller enters pthread_cond_wait() while holding the mutex. The function **atomically releases the mutex and puts the thread to sleep**. When it wakes, it reacquires the mutex before returning.

```

Consumer:

pthread_mutex_lock()
|

```

```

      v
queue empty?
      |
      yes
      |
      v
pthread_cond_wait()
      |
      +--> atomically release mutex
      |
      +--> sleep
      .
      .
producer inserts data
      |
pthread_cond_signal()
      |
consumer wakes
      |
reacquire mutex
      |
pthread_cond_wait() returns
      |
recheck condition
      |
pop element

```

The atomic release-and-wait behavior is critical. If unlocking and sleeping were separate operations, a producer could signal between those two operations and the consumer could miss the event - a classic lost-wakeup race.

8. Why must condition waits use while instead of if?

Always recheck the predicate after waking:

```

while (q->count == 0)
    pthread_cond_wait(&q->not_empty, &q->lock);

```

There are two main reasons. First, condition variables permit **spurious wakeups**. Second, with multiple consumers, another consumer may wake and acquire the mutex first, consume the available element, and make the queue empty again before this thread obtains the mutex. Therefore a wakeup means only 'the condition may now be true,' not 'the condition is definitely true.'

9. signal() versus broadcast()

pthread_cond_signal() wakes one waiter and is normally enough when a push creates one newly consumable item or a pop creates free space. **pthread_cond_broadcast()** wakes all waiters and is useful when a state transition can allow many waiters to proceed, or during shutdown. Waking everyone unnecessarily can cause the thundering-herd problem.

10. Blocking versus non-blocking FIFO APIs

A blocking push waits when the queue is full; a blocking pop waits when it is empty. A non-blocking try_push/try_pop returns immediately with failure if the operation cannot currently proceed. Which API is correct depends on the caller and latency requirements.

11. Complexity and performance

Both push and pop are O(1). No element shifting is required. The circular buffer also avoids per-operation dynamic allocation. Under contention, however, the mutex serializes queue modifications. More advanced designs may use semaphores, atomics, or lock-free queues, but they add complexity and should be justified by workload requirements.

12. Complete bounded userspace FIFO

```
#include <stdio.h>
#include <pthread.h>
#include <stdbool.h>

#define FIFO_SIZE 8

typedef struct {
    int buffer[FIFO_SIZE];
    int head;
    int tail;
    int count;

    pthread_mutex_t lock;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
} fifo_t;

int fifo_init(fifo_t *q)
{
    if (!q)
        return -1;

    q->head = q->tail = q->count = 0;

    if (pthread_mutex_init(&q->lock, NULL) != 0)
        return -1;

    if (pthread_cond_init(&q->not_empty, NULL) != 0) {
        pthread_mutex_destroy(&q->lock);
        return -1;
    }

    if (pthread_cond_init(&q->not_full, NULL) != 0) {
        pthread_cond_destroy(&q->not_empty);
        pthread_mutex_destroy(&q->lock);
        return -1;
    }

    return 0;
}

int fifo_push(fifo_t *q, int value)
{
    if (!q)
        return -1;

    pthread_mutex_lock(&q->lock);

    while (q->count == FIFO_SIZE)
        pthread_cond_wait(&q->not_full, &q->lock);

    q->buffer[q->tail] = value;
    q->tail = (q->tail + 1) % FIFO_SIZE;
    q->count++;

    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
    return 0;
}

int fifo_pop(fifo_t *q, int *value)
{
    if (!q || !value)
        return -1;

    pthread_mutex_lock(&q->lock);

    while (q->count == 0)
        pthread_cond_wait(&q->not_empty, &q->lock);

    *value = q->buffer[q->head];
    q->head = (q->head + 1) % FIFO_SIZE;
    q->count--;
}
```

```

        pthread_cond_signal(&q->not_full);
        pthread_mutex_unlock(&q->lock);
        return 0;
}

bool fifo_try_push(fifo_t *q, int value)
{
    if (!q)
        return false;

    pthread_mutex_lock(&q->lock);

    if (q->count == FIFO_SIZE) {
        pthread_mutex_unlock(&q->lock);
        return false;
    }

    q->buffer[q->tail] = value;
    q->tail = (q->tail + 1) % FIFO_SIZE;
    q->count++;

    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
    return true;
}

bool fifo_try_pop(fifo_t *q, int *value)
{
    if (!q || !value)
        return false;

    pthread_mutex_lock(&q->lock);

    if (q->count == 0) {
        pthread_mutex_unlock(&q->lock);
        return false;
    }

    *value = q->buffer[q->head];
    q->head = (q->head + 1) % FIFO_SIZE;
    q->count--;

    pthread_cond_signal(&q->not_full);
    pthread_mutex_unlock(&q->lock);
    return true;
}

void fifo_destroy(fifo_t *q)
{
    if (!q)
        return;

    pthread_mutex_destroy(&q->lock);
    pthread_cond_destroy(&q->not_empty);
    pthread_cond_destroy(&q->not_full);
}

```

13. Producer-consumer example

```

fifo_t fifo;

void *producer(void *arg)
{
    int id = *(int *)arg;

    for (int i = 0; i < 10; i++) {
        int value = id * 100 + i;
        fifo_push(&fifo, value);
        printf("Producer %d -> %d\n", id, value);
    }

    return NULL;
}

void *consumer(void *arg)

```

```

{
    int id = *(int *)arg;

    for (int i = 0; i < 10; i++) {
        int value;
        fifo_pop(&fifo, &value);
        printf("Consumer %d <- %d\n", id, value);
    }

    return NULL;
}

```

14. What changes in the Linux kernel?

The same logical FIFO state still needs synchronization, but kernel primitives depend on execution context. A common process-context character-driver design uses a **spinlock** for short non-sleeping state updates and **wait queues** to block readers or writers.

15. Mutex versus spinlock

A userspace pthread mutex may put the waiting thread to sleep. In the kernel, a spinlock is intended for very short critical sections and the holder must not sleep. A thread trying to acquire a contended spinlock spins until it becomes available. Therefore the protected region must be small.

Use a spinlock when the same state may need protection in atomic/interrupt-related contexts, or where a short non-sleeping critical section is appropriate. If the entire operation runs in sleepable process context, a kernel mutex can often be simpler. The primitive must match context.

16. Why spin_lock_irqsave()?

If the same lock-protected data can also be touched from interrupt context on the local CPU, merely taking the spinlock in process context can deadlock if an interrupt arrives and its handler tries to acquire that same lock. `spin_lock_irqsave()` acquires the lock while saving and disabling local interrupt state; `spin_unlock_irqrestore()` restores it. Do not use `irqsave` mechanically - use it when the locking/context design requires it.

17. Kernel wait queues

A wait queue lets a task sleep until a condition becomes true. A reader can wait until `count > 0`; a writer can wait until `count < FIFO_SIZE`. The condition must be rechecked because wakeups do not grant ownership of the resource.

```

Reader:
wait_event_interruptible(read_queue, count > 0);

Writer:
wait_event_interruptible(write_queue, count < FIFO_SIZE);

After write:
wake_up_interruptible(&read_queue);

After read:
wake_up_interruptible(&write_queue);

```

18. Why must we not hold a spinlock around copy_to_user()?

`copy_to_user()` and `copy_from_user()` access userspace memory. That access may fault and may require sleeping while the fault is resolved. Sleeping while holding a spinlock is illegal. Therefore user copies must be outside the spinlocked critical section.

```

WRONG:

spin_lock(&lock);
copy_to_user(...);      /* may sleep */
spin_unlock(&lock);

```

BETTER STRUCTURE:

```
spin_lock(&lock);  
copy FIFO data to kernel temporary buffer;  
update FIFO metadata;  
spin_unlock(&lock);  
  
copy_to_user(...);          /* no spinlock held */
```

19. Important failure-semantics subtlety

The simplified structure above introduces another issue: if bytes are removed from the FIFO and then `copy_to_user()` fails, those bytes have already been consumed. Production code must deliberately define how partial/faulting copies affect FIFO state. This is exactly the sort of follow-up an experienced kernel interviewer may ask.

20. READ_ONCE and wait conditions

A wait condition may be evaluated without holding the FIFO lock. `READ_ONCE()` prevents the compiler from inventing or combining accesses to that shared scalar in ways inappropriate for this polling-style observation. It does not replace the lock for compound state changes, and it is not by itself a general synchronization protocol.

21. Why not use volatile?

`volatile` is not a thread-synchronization primitive. It does not provide mutual exclusion, atomic read-modify-write behavior, or the required inter-thread memory-order guarantees. Use mutexes, spinlocks, atomics, and the kernel/user synchronization APIs designed for concurrency.

22. Memory ordering

Locks do more than prevent two threads from entering a critical section simultaneously. Correct lock operations also provide ordering guarantees so that writes performed before unlocking become appropriately visible to a thread that later acquires the same lock. This is why replacing a lock with ordinary loads/stores or `volatile` accesses is incorrect.

23. Multiple producers and multiple consumers

The mutex-protected userspace design supports multiple producers and consumers because every queue-state transition is serialized. After a wakeup, each waiter competes for the mutex and then rechecks the predicate. This is another reason the condition test must be a while loop.

24. What about shutdown?

A production blocking queue normally needs a closed/stopping flag. Otherwise a thread can remain blocked forever in `fifo_pop()` after producers have terminated. Shutdown typically sets the flag while holding the lock and broadcasts to waiting threads; each waiter wakes, checks both the queue condition and the shutdown state, and exits cleanly when appropriate.

25. Semaphores as an alternative

A bounded producer-consumer queue can also use two counting semaphores: one representing available items and one representing available slots, plus a mutex for the actual buffer mutation. Initially `items = 0` and `slots = FIFO_SIZE`. A producer waits on slots, inserts under the mutex, then posts items. A consumer waits on items, removes under the mutex, then posts slots.

26. Lock-free FIFO?

Lock-free queues can reduce some forms of blocking and contention, but correctness becomes substantially harder: atomic operations, memory ordering, ABA issues, reclamation, and single-producer/single-consumer versus

multi-producer/multi-consumer assumptions all matter. In an interview, start with the correct mutex/condition-variable solution unless lock-free behavior is explicitly required.

27. Common interview bugs

- Checking empty/full outside the lock and then acting on stale state.
- Using if instead of while around a condition-variable wait.
- Forgetting to wrap head/tail.
- Updating count outside the protected critical section.
- Busy-waiting instead of blocking when that is not required.
- Calling a sleeping operation while holding a kernel spinlock.
- Assuming volatile makes a queue thread-safe.
- Destroying a mutex/condition variable while threads are still using or waiting on it.
- Forgetting shutdown semantics for blocked threads.

28. Interview answer in 30 seconds

A strong concise answer is: *I would implement a bounded FIFO as a circular buffer with head, tail, and count. A mutex protects all queue state. Consumers wait on a not_empty condition while count is zero; producers wait on not_full while count equals capacity. Condition waits are inside while loops because of spurious wakeups and competition among waiters. Push and pop are O(1). In a Linux driver, I would use the appropriate kernel lock plus wait queues, keep spinlocked regions short, and never call copy_to_user/copy_from_user while holding a spinlock.*

29. Follow-up questions to prepare

- What happens if two consumers wake for one element?
- Why is pthread_cond_wait atomic with respect to releasing the mutex?
- What is a lost wakeup?
- Why use while instead of if?
- signal versus broadcast?
- What if the FIFO is full and the producer must not block?
- Why not volatile?
- Mutex versus spinlock?
- Can copy_to_user sleep?
- Why might irqsave be necessary?
- How would you support poll/select/epoll in a character driver?
- How would you implement clean shutdown?
- How would you change this for a single-producer/single-consumer queue?
- When would Linux kfifo be preferable to implementing the ring manually?